PRESENTAZIONE TECNICA

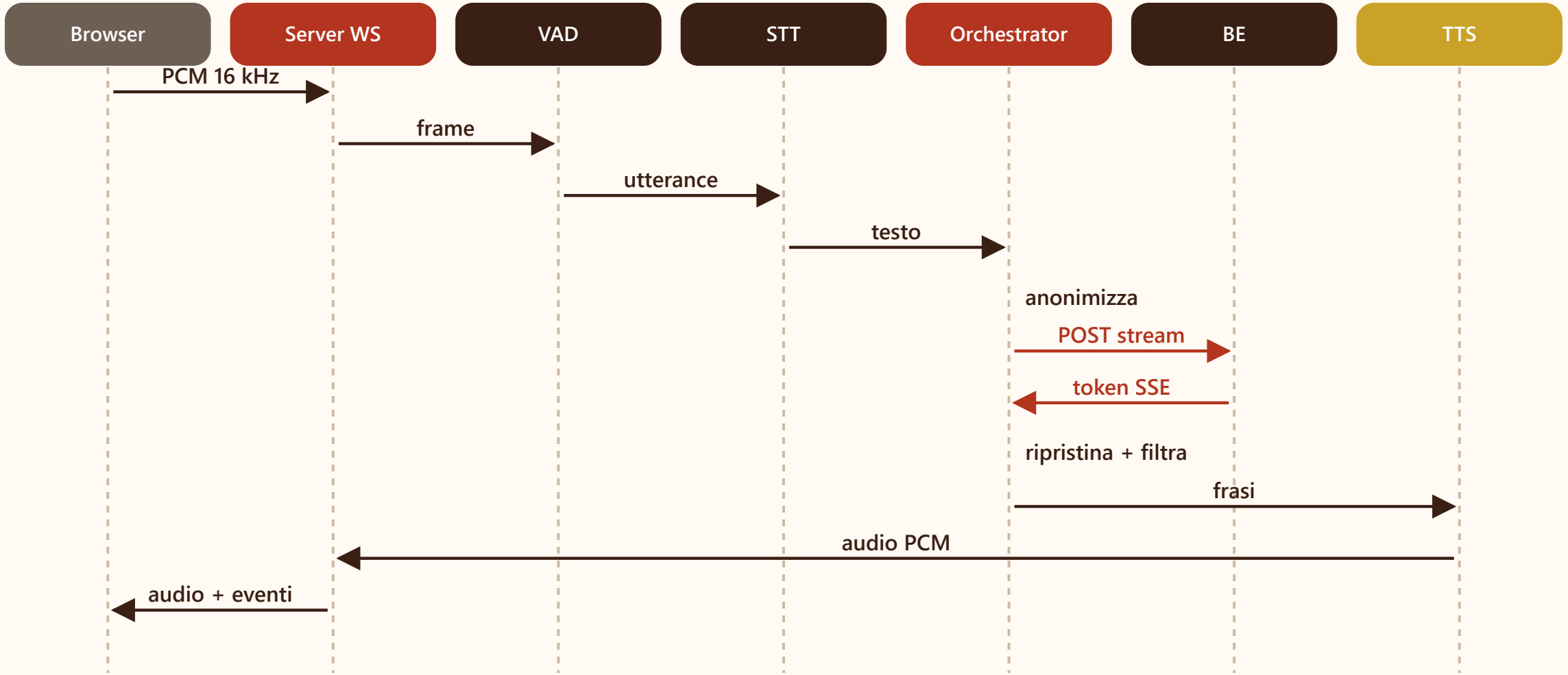
Il flusso del codice Voice

Dal microfono del browser alla risposta parlata: moduli, code, stati, privacy e punti di contatto con il BE

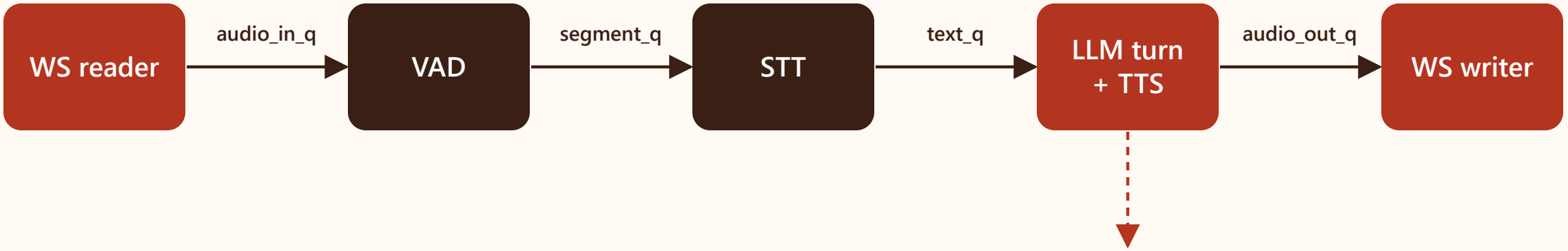
Mappa del codice: 5 livelli

Trasporto	pipeline/server.py (FastAPI, WebSocket, /site/*) · pipeline/session.py (VoiceSession: le 3 fasi) __main__.py (comandi)
Audio	vad.py (energia / webrtc) · stt.py (faster-whisper) tts.py (Piper) · audio.py (PCM, frasi, speakable)
Conversazione	orchestrator.py · contacts.py (registrazione) · anonymizer.py + pii.py · toolguard.py messages.py · prompts.py · spoken.py
Verso il BE	backend.py (httpx, retry, SSE) chat_service.py (alternativa: passa dal servizio CHAT)
Strumenti	doctor.py + diagnostics.py · local.py + devtools/stub_be.py · live_client.py v2.py · config.py · logging_json.py

Una chiamata, dall'orb al BE e ritorno



VoiceSession: tre fasi collegate da code



Ogni coda è LIMITATA (backpressure): se il server è in ritardo si perde il frame più vecchio, mai memoria che cresce

Barge-in (interruzione)

parlato ≥ 300 ms mentre il bot parla → annulla il turno, svuota l'audio in coda, avvisa il client

Contatore di generazione

ogni interruzione incrementa _gen: il lavoro «vecchio» si accorge di essere scaduto e si ferma

VAD: quando una persona ha finito di parlare



Regole

soglia energia 500 · pausa 600 ms · voce minima 250 ms
segmento massimo 15 s (poi taglia comunque)

Il bug che abbiamo risolto

il tempo si misura sull'audio ricevuto, non sul numero di frame: il browser manda 128 ms alla volta, il client Python 20 ms → prima la pausa era 6 volte troppo lunga

Due classificatori, una sola macchina a stati: EnergyVAD (default, zero dipendenze) e WebRtcVADWrapper (opzionale)

Riconoscimento e sintesi: scelte e trucchi

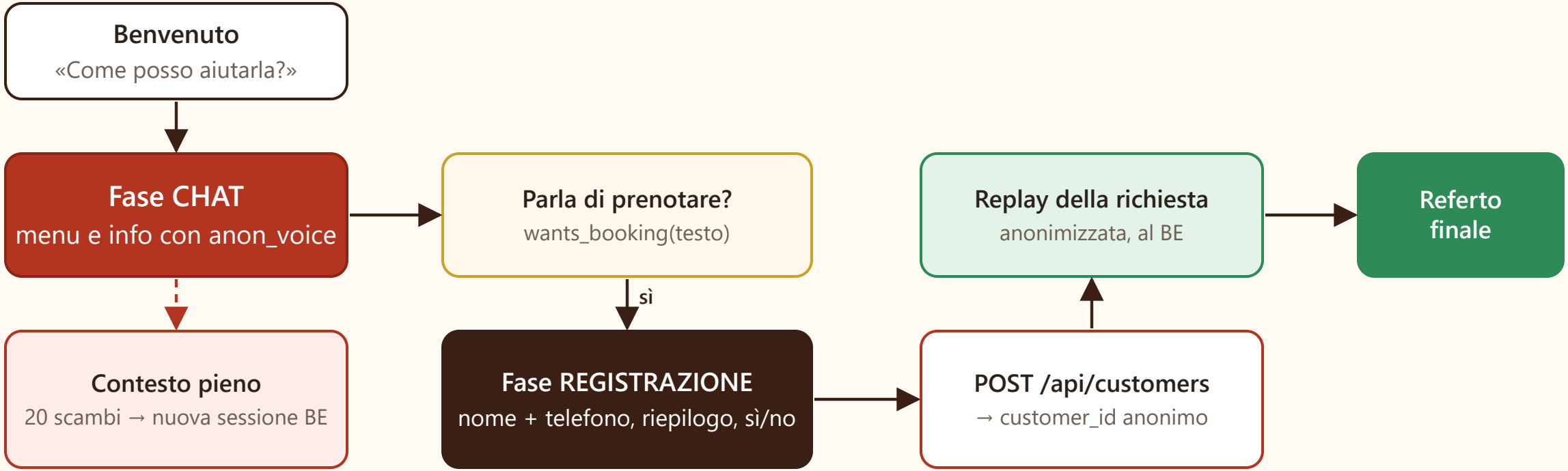
STT · faster-whisper

- Modello **small**, int8, CPU: caricato **una volta** e condiviso da tutte le sessioni
- `beam_size=1`, lingua dall'.env (italiano), `vad_filter=False` (il VAD è già nostro)
- Scarta i segmenti con `no_speech_prob ≥ 0.6` e le «allucinazioni» note di Whisper
- Un **lock**: una trascrizione alla volta; gira in un thread per non bloccare l'event loop
- Prompt iniziale **spento** di default: veniva ripetuto al posto del nome

TTS · Piper

- Voce **it_IT-riccardo-x_low** (onnxruntime), frequenza annunciata al client nell'evento `ready`
- **SentenceChunker**: i token del BE diventano frasi complete e Piper parla **appena la prima è pronta**
- `speakable()`: via markdown ed emoji; i codici diventano «R E S, 9 A 8 B…»
- `spoken.py`: numeri, date e orari a parole («alle venti e trenta»)
- API di Piper ≥ 1.3 gestita (la 1.8 installata rompeva il TTS)

Orchestrator: le fasi della conversazione



Il BE decide tutto il resto (strumenti, conferma sì/no, allergie). Il voice decide solo QUANDO chiedere i dati di contatto.

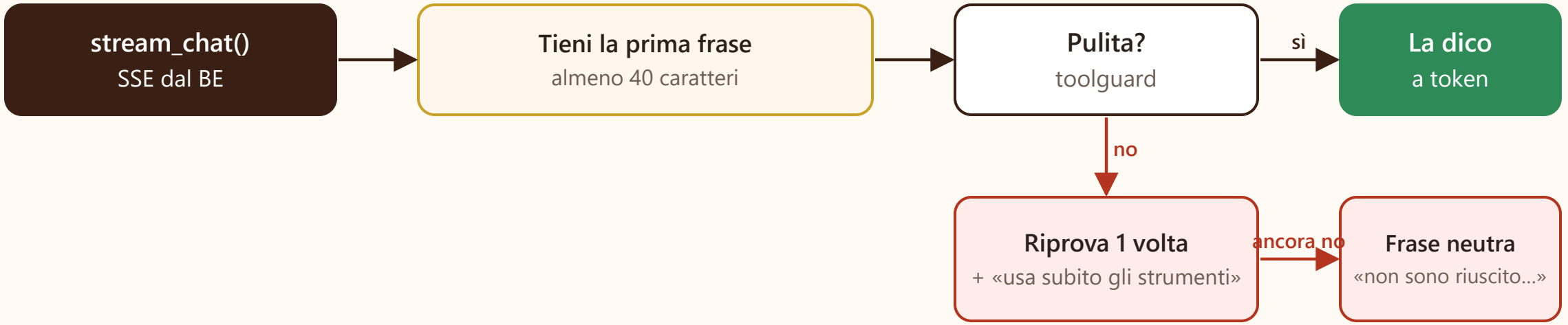
Privacy: dove vanno i dati personali

Dato	Da dove nasce	Dove può andare	Cosa vede l'LLM
Nome e telefono	dette a voce in fase REGISTRAZIONE	**solo** `POST /api/customers` → tabella *customers*	nulla: vede solo `cust_…`
Frasi libere	es. «sono Mario, il mio 333…»	`anonymize()` → `[NOME_1]`, `[TELEFONO_1]` → BE	segnaposto opachi
Risposta del BE	può contenere segnaposto	`restore()` prima della voce (mai mezzo segnaposto detto)	—
Ultima rete	tutto ciò che esce verso il BE	`scrub_pii()`: regex senza stato su e-mail, telefono, CF, indirizzo	—
Log	telefono nei log	mascherato: solo i primi 4 caratteri + «***»	—

Strati dell'anonimizzatore

- 1) regex per forme fisse (e-mail, CF, telefono, indirizzo)
- 2) regex per le frasi che introducono un nome («a nome di…»)
- 3) spaCy `it_core_news_md` come rete di sicurezza (opzionale)

Il turno col BE: il filtro sulle risposte



Cosa intercetta il filtro

nomi di strumenti (get_menu…) · «devo prima chiamare…» · «vuole che verifichi…?» · «Posso mostrarle i piatti…» detto al posto di mostrarli

Referto finale e conferma vocale

Il BE conferma
nel testo c'è RES-XXXXXX

→

GET /api/reservations/{code}
solo il proprietario (altrimenti 404)

→

Referto detto A VOCE
costruito dal codice, non dall'LLM

«Sì» breve (≤ 6 parole, senza «ma»/numeri)
→ evento reservation_confirmed → il popup si chiude dopo 4 s

«Sì, ma alle 21»
NON è una conferma: va al BE come richiesta di modifica

«No»
«Mi dica cosa vuole cambiare»

Eventi verso la pagina
reservation (nome, data, ora, persone, ambiente se chiesto)
registered · reservation_confirmed · call_ended

Contesto, cache e prestazioni

PROBLEMA

Il BE tiene tutto il dialogo come contesto dell'LLM: con un modello locale su CPU più contesto = più lento; il BE inoltre scrive data e `customer_id` in cima al prompt, quindi ogni nuovo id fa rileggere tutto.

COSA FA IL VOICE

- **20 scambi** senza arrivare a una prenotazione → nuova sessione BE + recap parlato
- Al **3° ripristino** avvisa («potrei dover ricominciare»); al **4°** chiude la chiamata (evento `call_ended`)
- Prenotazione raggiunta → contatori a zero; dopo il «sì» finale sessione nuova
- Cliente resta registrato: nessuno ripete nome e telefono

Prestazioni su portatile

- **Warm-up** all'avvio: un turno finto al BE, così il primo cliente non aspetta ~1 minuto
- Id anonimo **condiviso** `anon_voice`: stessa cache del prompt per tutti gli anonimi
- Modello con contesto 8k e 6 thread (`voice/local_llm/Modelfile`)
- Misurato: primo turno ~41–70 s, poi 1–5 s

Protocollo WebSocket /ws/voice

Verso	Contenuto	Note
↑ binario	PCM16 mono 16 kHz	browser: blocchi da 2048 campioni (128 ms) · client Python: 20 ms
↑ testo	{"cmd":"eos"} · {"cmd":"reset"}	fine flusso · «nuova conversazione»
↓ binario	PCM16 al tts_sample_rate (22050)	l'audio del bot, a pezzi, frase per frase
↓ eventi	loading · ready · heard · transcript · thinking · reply · barge_in	stati della chiamata per l'interfaccia
↓ eventi	registered · reservation · reservation_confirmed · call_ended · error	fatti strutturati: popup, «Le mie prenotazioni», fine chiamata

Indirizzo: `ws://HOST:8003/ws/voice` · timeout di inattività 60 s · un solo socket per chiamata.

Il sito e l'interfaccia

Pagine servite dal server voice

- `/` il sito: menu, info, «Le mie prenotazioni», orb, chat
- `/test-voice` pagina di test con tab TECNICA / STATO, trascrizione ed eventi
- `/site/menu · info · reservation/{codice}`: il browser non parla mai col BE, passa da qui
- `/site/chat/*` → servizio CHAT (spento qui)
- `/health · /ready · /diag`

L'orb parla da solo

- **Pallini bianchi** che vanno al centro = il BE sta pensando
- **Anello dorato** sottile e lento = a riposo
- Anello che **si ispessisce** con la fascia chiara che cresce = sta per parlare
- Anello che **ondeggia** (non in sincrono con l'orb) = sta parlando
- **Popup** di riepilogo: si chiude 4 s dopo il «sì» finale, solo a voce

Resilienza: cosa succede quando qualcosa va storto

Situazione	Cosa facciamo	Perché
BE irraggiungibile	retry con backoff esponenziale	se fallisce: frase parlata «non riesco a raggiungere il server»
Timeout in lettura su POST	**mai ritentato**	la richiesta potrebbe essere già stata eseguita: niente doppioni
Stream interrotto	riconnessione solo prima del primo token	dopo, si chiude: riprovare ripeterebbe parole
Rete del client	live_client / pagina: riconnessione con backoff	la chiamata sopravvive a una caduta
Modelli non pronti	evento `loading`, attesa fino a 120 s	il caricamento parte all'avvio (PRELOAD_MODELS)
Errori tecnici	mai letti ad alta voce	sempre una frase breve e gentile